Artificial Intelligence — Assignment 5

Markov Decision Processes: Policy Evaluation, Value Iteration & Policy Iteration

Course MA3206 AI

Group N

Aryan Vats (2301MC52) | Kushal Kesharwani (2301MC57) | Aditya Aryan (2301MC58) | Pranshu Deep (2301MC59)

MDP Setup

The problem models a robot with three battery states — **High, Low,** and **Charging** — and two actions: **Search** and **Wait**. The discount factor is $\gamma = 0.9$ and the convergence threshold is $\theta = 1\times10^{-6}$. The transition and reward structures were encoded as NumPy arrays P (shape 3×2×3) and R (shape 3×2). All row sums of P were verified to equal 1.0, confirming a valid probability model.

State	Action	Next State	P(s'|s,a)	R(s,a)
High	Search	High / Low	0.7 / 0.3	+4
High	Wait	High	1.0	+1
Low	Search	High / Low	0.4 / 0.6	–3 / +4 (exp: +1.2)
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

Task 1 — Iterative Policy Evaluation

1.1 Policy Evaluated

The fixed policy evaluated was: **High** → **Search, Low** → **Wait, Charging** → **Wait**. The Bellman expectation update was applied repeatedly until the maximum change across all states fell below θ.

1.2 Results

Policy evaluation converged in **133 iterations**. The converged state values are:

State	$V_\pi(s)$
High	18.1081
Low	9.9999
Charging	16.2973

The High state yields the greatest value because Searching from High always gives +4 reward with no penalty risk. The Low state has the smallest value under Wait (+1 each step), and Charging sits between the two since it transitions deterministically back to the high-value High state.

1.3 Bar Chart — Vπ(s)

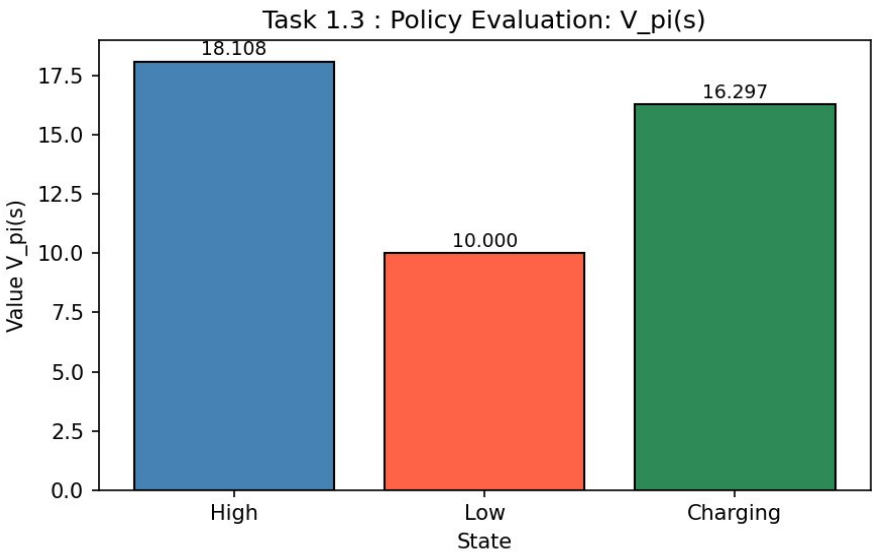

Figure 1: Policy evaluation values for the fixed policy [Search, Wait, Wait].

Task 2 — Value Iteration

2.1 / 2.2 Optimal Values and Policy

Value iteration applies the Bellman optimality update — taking the max over actions at every sweep — until convergence. It converged in **142 iterations**. The optimal values and the extracted greedy policy are:

State	V*(s)	Optimal Action
High	29.6438	Search
Low	25.8082	Search
Charging	26.6794	Wait

Notably, the optimal policy prescribes **Search in both High and Low** states. While Low-Search carries a 0.4 probability of a –3 penalty, the long-run discounted value of returning to High (worth ~29.6) outweighs the immediate risk when the agent is already accumulating high-value rewards. Wait in Charging is the only valid action.

2.3 Convergence Plot

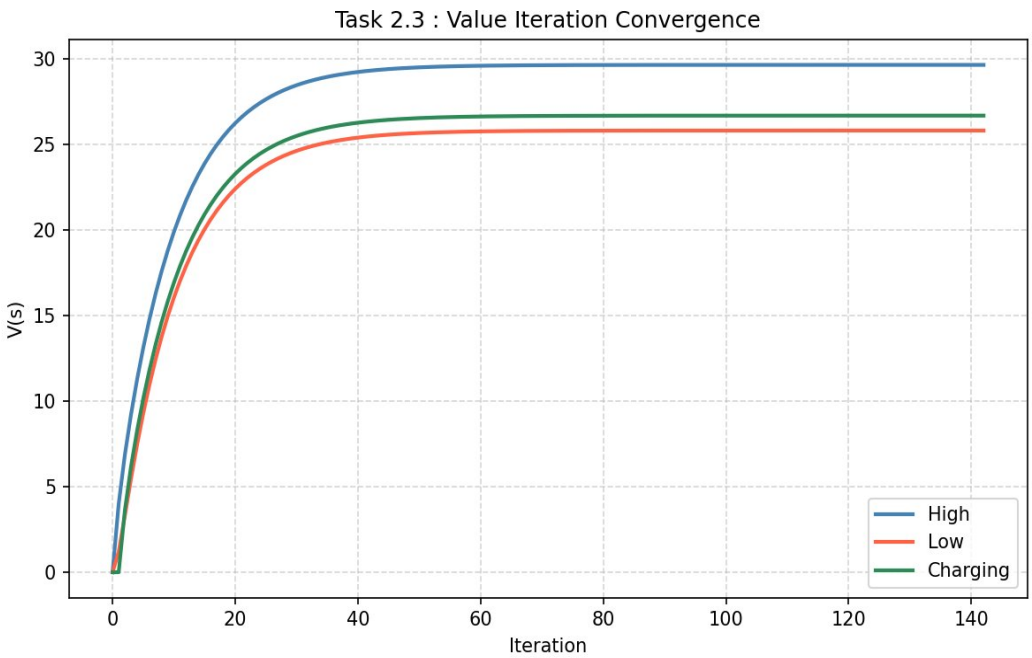

Figure 2: V(s) across 142 value iteration sweeps. All states start at 0 and rise monotonically.

Task 3 — Policy Iteration

3.1 / 3.2 Algorithm and Results

Policy Iteration alternates between full policy evaluation and a greedy policy improvement step. Starting from an all-Wait policy, it converged in just **2 outer iterations** — far fewer than value iteration's 142 sweeps. The policy history is shown below.

State	Iteration 1 (initial)	Iteration 2 (final)
High	Wait	Search
Low	Wait	Search
Charging	Wait	Wait

Both algorithms converge to the same optimal policy: **Search in High and Low, Wait in Charging**. The final V* values from policy iteration exactly match those from value iteration.

3.3a V(s) Across Policy Iteration Steps

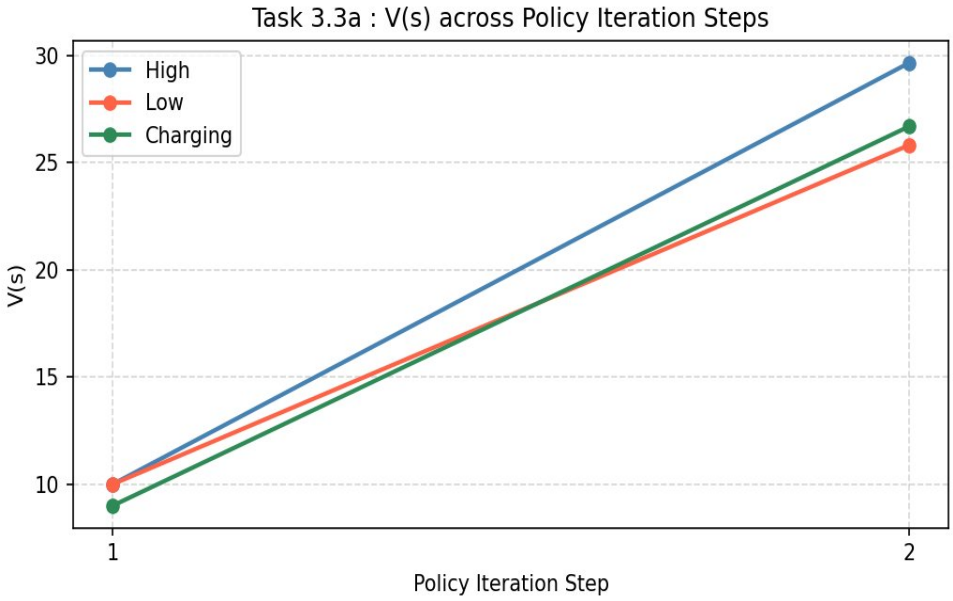

Figure 3: V(s) jumps sharply from iteration 1 to 2 as the policy switches from all-Wait to the optimal policy.

3.3b Policy Heatmap

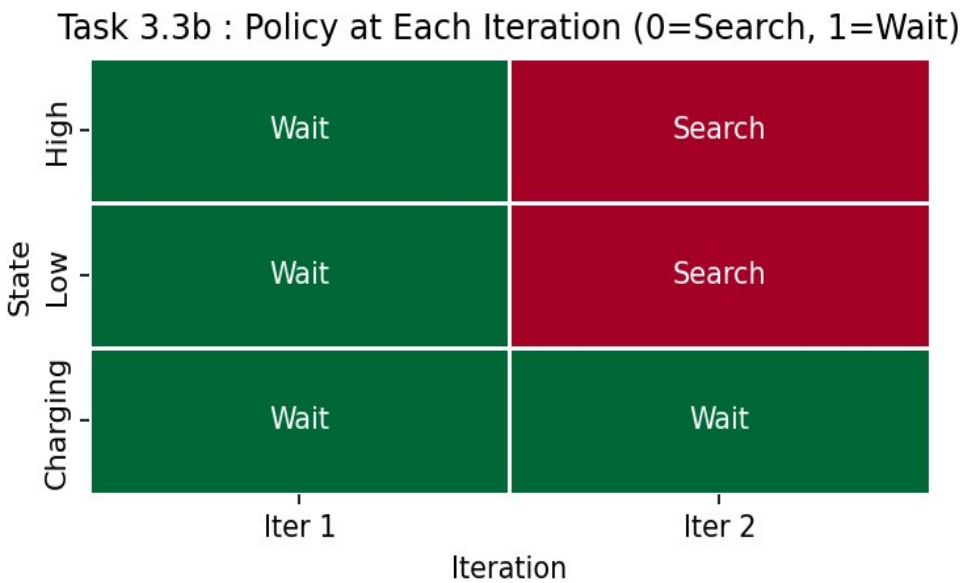

Figure 4: Policy heatmap showing action per state at each iteration. Green = Wait, Red = Search.

Task 4 — Analysis and Interpretation

4.1 Value Iteration vs. Policy Iteration

Policy Iteration converges in 2 outer loops while Value Iteration requires 142 sweeps. In Policy Iteration, each evaluation step fully solves for V^{π} before the improvement step, so the policy jumps directly to the optimum in a single improvement. Value Iteration, by contrast, only partially backs up values each sweep and must gradually propagate discounted future rewards through all states before the greedy extraction yields the correct policy. For small MDPs like this one, Policy Iteration is clearly more efficient in terms of outer iterations, though each outer iteration internally runs a full evaluation.

4.2 Convergence Behavior

In both algorithms, V(s) starts at 0 and increases monotonically. The High state rises fastest because Search always yields +4 and there is no recharge penalty. Low and Charging states lag slightly but also converge upward. The key difference from Policy Evaluation: Value Iteration takes the max over actions at every step, so it is never constrained to a suboptimal action — this means it typically needs more iterations to settle fine-grained differences between action values, but it does not depend on an external policy.

4.3 Optimal Policy Interpretation

The optimal policy is **Search in High, Search in Low, and Wait in Charging**. For the High state, Searching always yields +4 with no recharge penalty, making it the clear dominant action. For the Low state, although Searching risks a –3 penalty (with probability 0.4), there is also a 0.4 chance of transitioning back to High — which is worth ~29.6 discounted. The expected long-run return from Search exceeds the safe +1 from Wait. Charging has only one valid action (Wait), which transitions the robot to High for free.

4.4 Practical Insight

In real-world battery-operated systems, a state-aware policy like this is valuable because it avoids both overly conservative behavior (waiting indefinitely when the robot could be productive) and reckless behavior (searching until the battery dies). The policy naturally encodes risk management: act aggressively when resources are adequate, and accept short-term constraints when the situation calls for it. This kind of learned policy can also adapt if the transition probabilities or rewards change, making it more robust than hard-coded rule-based systems.